



ODO-101

Odometri: Teoriden Gerçek Zamanlı C++ Gerçeklemesine

Otonom Sürüş Birimi — Bireysel Gelişim Ödevi

Birim: BURST — Otonom Sürüş & Algı

Süre: 6 hafta

Dil: C++17 (yalnızca standart kütüphane)

Teslim: Git deposu + el yazısı teori defteri + sözlü kod savunması

Tarih: 24 Ağustos 2026

*“Aracın nerede olduğunu bilmeyen hiçbir planlayıcı, nereye gideceğine karar veremez.
Odometri, otonom sürüşün saat gibi işleyen kalbidir.”*

1 Ödevin Amacı

Bu ödevin sonunda; bir otonom aracın **tekerlek enkoderleri**, **IMU** ve **direksiyon açısı** verisinden kendi konumunu nasıl kestirdiğini yalnızca *anlatabilen* değil, tamamını **sıfırdan**, **elle**, **C++ ile yazabilen** bir takım üyesi olman hedefleniyor.

Bu bir “kütüphane kullanma alıştırması” **değildir**. Eigen yok, OpenCV yok, hazır Kalman sınıfı yok. Matris çarpımını da sen yazacaksın, açı normalizasyonunu da. Amaç; AWSIM simülatoründe ve gerçek araçta karşımıza çıkan `/odom`, `odom → base_link` TF zinciri, enkoder sürüklenmesi (drift), jiroskop bias’ı gibi kavramların senin için kara kutu olmaktan çıkması.

Neden odometri? BURST aracında şerit takibi (ZED kamera), BEV dönüşümü ve planlama katmanlarının tamamı *araç pozunun* doğru kestirimine yaslanır. GPS’in olmadığı (tünel, kapalı pist) veya kameranın kör kaldığı (güneş yansıması, şerit silikliği) her an, araç yalnızca odometriyle “ayakta kalır”. Odometri, otonom yığının sigortasıdır.

2 Demir Kurallar

1. **Her satır elle yazılacak.** Kopyala-yapıştır kod, yapay zekâya yazdırılmış kod, “internetten uyarlanmış” kod **yasak**. İnternetten *fikir* öğrenmek serbest; öğrendiğini kapat, kendin yaz.
2. **Harici kütüphane yok.** Yalnızca C++17 standart kütüphanesi (`<cmath>`, `<vector>`, `<array>`, `<random>`, `<fstream>` vb.). Matris sınıfını, açı aritmetiğini, filtreyi sen yazacaksın.
3. **Kod savunması.** Teslimden sonra mentorun kodundan *rastgele* satırlar seçer; “bu satır ne yapıyor, neden var, silersek ne bozulur?” sorularına takılmadan cevap veremediğin her satır puan düşürür. Anlamadığın satırı yazmayacaksın — yazamazsın zaten, çünkü sen yazdın.
4. **Her formül önce kâğıtta.** Koda dökülen her denklem (kinematik model, kovaryans yayılımı, Kalman kazancı) önce **el yazısı teori defterinde** türetilmiş olacak. Defter teslimatın parçasıdır.
5. **Git disiplini.** Her görev ayrı commit; mesajlar anlamlı (`feat(gorev2): ackermann kinematik modeli + birim testi`). Tek dev commit atan, ödevi baştan yazar.
6. **Derlenmeyen kod teslim edilmez.** `g++ -std=c++17 -Wall -Wextra` ile sıfır uyarı hedeflenir.

3 Yol Haritası (6 Hafta)

Hafta	Teori	Pratik / Kod
1	2B rijit cisim hareketi, poz kavramı, $SE(2)$, açı sarması ($[-\pi, \pi]$)	Görev 0: Pose2D, Vec2, açı normalizasyonu + birim testleri
2	Enkoder fiziği, tick $\rightarrow$ mesafe, diferansiyel sürüş kinematığı	Görev 1: enkoder simülatörü + ölü hesap (dead reckoning), Euler vs. tam yay entegrasyonu
3	Ackermann / bisiklet modeli, direksiyon geometrisi, kalibrasyon hataları	Görev 2: bisiklet modeli odometrisi; yarıçap-/dingil hatasının drift'e etkisi deneyi
4	Olasılıksal hata modeli, kovaryans, Jacobian ile belirsizlik yayılımı	Görev 3: Monte Carlo simülasyonu, kovaryans elipsi çıktısı (CSV $\rightarrow$ grafik)
5	Kalman filtresi: önce 1B sezgi, sonra EKF türetimi	Görev 4: elle Mat3 + 1B KF + $[x, y, \theta]$ durumlu EKF
6	Sensör füzyonu mimarisi, IMU bias, zaman damgası hizalama	Görev 5: enkoder+jiroskop füzyonu; sensör araştırma raporu; savunma provası

Sıra bilinçli: her hafta bir öncekinin üstüne inşa eder. Hafta 2'yi oturtmadan hafta 5'e atlarsan Kalman'da kaybolursun; filtre, modelin ne olduğunu bilmeyene hiçbir şey söylemez.

4 Teorik Temel — Defterde Türetilcekler

4.1 Poz ve açı aritmetiği

Araç pozunu $\mathbf{x} = (x, y, \theta)$. Defterde: dönme matrisi $R(\theta)$ 'nin neden ortogonal olduğu, iki pozun bileşkesi ($\oplus$ operatörü) ve açı farkının neden *sarmalanmak* (wrap) zorunda olduğu — 350° ile 10° arasındaki fark 340° değil 20° 'dir; bunu yanlış yapan filtre patlar.

4.2 Enkoderden harekete

N tick/devir çözünürlüklü enkoder, R yarıçaplı tekerlek:

$$\Delta s = \frac{2\pi R}{N} \Delta \text{tick}$$

Diferansiyel model (b : iz genişliği):

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta \theta = \frac{\Delta s_R - \Delta s_L}{b}$$

Ackermann / bisiklet modeli (L : dingil mesafesi, δ : direksiyon açısı):

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \frac{v}{L} \tan \delta$$

Defterde: Euler entegrasyonu ile tam yay (exact arc) entegrasyonunun türetimi ve hangi koşulda farkın büyüdüğü (ipucu: keskin viraj + büyük Δt).

4.3 Hata nereden gelir?

Sistematik hatalar (yanlış R , yanlış b , lastik basıncı!) ile rastgele hatalar (kayma, tick kuantalama, jiroskop gürültüsü) ayrımı. Defterde: R 'de %1 hata 100 metrede kaç metre boyuna hata üretir? b 'de %1 hata tam tur sonunda kaç derece yön hatası üretir? **Sayıyla** göster.

4.4 Belirsizlik yayılımı ve EKF

Hareket modeli $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ için $F = \partial f / \partial \mathbf{x}$, $G = \partial f / \partial \mathbf{u}$ Jacobianları **elle** türetilecek. Kovaryans yayılımı:

$$P_{k+1} = F P_k F^\top + G Q G^\top$$

EKF düzeltme adımı (z : ölçüm, H : ölçüm Jacobianı):

$$K = P H^\top (H P H^\top + R_z)^{-1}, \quad \mathbf{x} \leftarrow \mathbf{x} + K(z - h(\mathbf{x})), \quad P \leftarrow (I - K H) P$$

Defterde her sembolün *fiziksel* anlamı bir cümleyle yazılacak. “ K kazançtır” yetmez; “ K , modele mi ölçüme mi güveneceğimizi belirsizlik oranından hesaplar” gibi.

5 C++ Görevleri

Tüm görevler tek depo içinde, `gorev0/ ...gorev5/` klasörlerinde. Her klasörde `main.cpp`, gerekiyorsa yardımcı başlıklar ve bir `README.md` (nasıl derlenir, ne çıkarır, ne öğrendin — 5 satır yeter).

Görev 0 — Temel Tipler (Isınma)

`Vec2`, `Pose2D` yapıları; `normalizeAngle(double)` fonksiyonu; iki pozu bileştiren `compose(a, b)`. Ardından kendi mini test çatın: `assert` ile en az 10 test (özellikle $\pm\pi$ sınırındaki açılar). **Çıktı:** tüm testler geçiyor; `normalizeAngle`'ın neden `fmod` ile tek satırda “bitmediğini” `README`'de açıkla.

Görev 1 — Enkoder Simülatörü + Ölü Hesap

Sanal bir araç sabit v ve ω ile daire çizsin. Enkoder simülatörün gerçek hareketi tick'lere *kuantalayarak* (tam sayı!) raporlasın. Tick'lerden pozu geri kur: önce Euler, sonra tam yay entegrasyonu. 100 tur sonunda iki yöntemin gerçek çembere göre hatasını CSV'ye dök. **Çıktı:** hata-zaman grafiği (Python/LibreOffice ile çizebilirsin; *grafik* için araç serbest, *hesap* için değil).

Görev 2 — Ackermann Odometrisi

Bisiklet modelini gerçekleştir: girişler v (enkoderden) ve δ (direksiyon). Sekiz çizen (fig-8) bir test rotası tanımla. Sonra **kasıtlı kalibrasyon hatası** enjekte et: R 'yi %2, L 'yi %3 yanlış ver. Drift'in rotaya ve hataya göre nasıl büyüdüğünü tablo halinde raporla. **Çıktı:** “hangi kalibrasyon hatası hangi tip drift üretir” cümlelik özeti — bunu savunmada soracağız.

Görev 3 — Gürültü ve Monte Carlo

`<random>` ile tick gürültüsü ve kayma modeli ekle. Aynı rotayı 1000 kez koştur; son pozların dağılımını CSV'ye yaz, ortalama ve 2×2 kovaryansı **elle** hesapla (hazır istatistik fonksiyonu yok). Kovaryans elipsinin neden hareket yönüne dik büyüdüğünü (yoksa paralel mi büyüyor?) gözlemleyip açıkla. **Çıktı:** saçılım grafiği + elle kovaryans hesabının kodu.

Görev 4 — Elle Matris + Kalman

Önce **Mat3**: çarpma, transpoz, 3×3 ters (kofaktör yöntemiyle, elle). Sonra 1B Kalman: sabit hızla giden araca gürültülü mesafe ölçümü — 30 satırlık filtrenin her satırını yorumla. En son $[x, y, \theta]$ durumlu EKF: öngörü adımı Görev 2'nin modeli, düzeltme adımı 1 Hz gelen gürültülü konum ölçümü (simüle GPS / AWSIM ground-truth gibi düşün). **Çıktı**: filtreli ve filtresiz yörüngeyi aynı grafikte karşılaştırması; Q ve R_z 'yi 10 kat büyütüp küçültünce ne olduğunun deney tablosu.

Görev 5 — Enkoder + Jiroskop Füzyonu

Jiroskop simülatörü: gerçek ω + beyaz gürültü + **yavaşça yürüyen bias**. EKF durumunu $[x, y, \theta, b_g]$ yap: bias'ı da kestir. Enkoder $\Delta\theta$ 'sı ile jiroskop $\Delta\theta$ 'sının tek başına ve füzyonla ürettiği yön hatasını karşılaştır. **Çıktı**: “jiroskop kısa vadede, enkoder uzun vadede” cümlesinin senin verinle ispatı.

Bonus — ROS 2 / AWSIM Entegrasyonu (+10)

Görev 5'teki filtreyi bir ROS 2 node'una taşı: `nav_msgs/Odometry` mesajı olarak `/odom` yayımla, REP-105'e uygun `odom` → `base_link` TF'i yayımla ve AWSIM'den gelen araç verisiyle besle. Burada `rc1cpp` serbesttir (ödevin özü olan matematik hâlâ senin kodun).

6 Sensör Araştırma Raporu (Zorunlu)

Odometri kadar, odometriyi **besleyen elektronik** de bizim işimiz. En fazla 4 sayfalık, kaynakçalı bir araştırma raporu yazacaksın:

- Enkoder teknolojileri**: optik vs. manyetik; artımsal vs. mutlak; kuadratür A/B sinyali yön bilgisini nasıl taşır? Çözünürlük (PPR) ile hız ölçüm bant genişliği ilişkisi. Mikrodenetleyiciye bağlantı: interrupt ile tick sayma vs. donanım sayacı (timer encoder modu) — hangisi neden?
- IMU sınıfları**: MEMS jiroskoplarda bias kararlılığı ($^{\circ}/h$) ne demek; tüketici sınıfı (MPU-6050 tarzı) ile endüstriyel sınıf arasında odometri açısından pratik fark nedir? Arayüzler: I2C vs. SPI vs. CAN — örnekleme hızı ve gecikme açısından karşılaştır.
- Araç tarafı**: otomobillerde tekerlek hız bilgisi ABS sensörlerinden CAN bus üzerinden gelir; bir CAN mesajından hız çözmenin adımlarını anlat. Direksiyon açısı sensörü nereden okunur?
- Görsel destek**: BURST'te kullandığımız ZED stereo kamera görsel odometri de üretebiliyor. Tekerlek odometrisiyle görsel odometrinin hata karakterleri neden farklıdır (kayma vs. ölçek/ı-şık)? Hangi durumda hangisi çöker?
- Seçim önerisi**: BURST aracı için “ben olsam şu enkoder + şu IMU + şu arayüzü seçerdim, çünkü...” paragrafı. Fiyat/performans dahil, gerekçesiz cümle yazma.

Rapor **kes-yapıştır kokusu** taşırsa kabul edilmez. Her kaynağı künyele; veri sayfasından (datasheet) okuduğun sayıyı sayfa numarasıyla göster.

7 Değerlendirme

Kalem	Ağırlık
El yazısı teori defteri (türetimler, sayısal örnekler)	%20
Kod: doğruluk, elle yazım, derleme temizliği, testler	%35
Sözlü kod savunması (rastgele satır sorgusu)	%20
Sensör araştırma raporu	%15
Deney raporları (CSV'ler, grafikler, yorumlar)	%10
Bonus: ROS 2 / AWSIM entegrasyonu	+%10

Geçme notu %70'tir. Kod savunmasından %50'nin altı alınırsa — kod kimin yazdığı belirsiz demektir — ödev bütünüyle tekrarlanır.

8 Kaynaklar

- Siegwart, Nourbakhsh, Scaramuzza — *Introduction to Autonomous Mobile Robots* (2. baskı): Bölüm 5, tekerlek kinematığı ve odometri hataları.
- Thrun, Burgard, Fox — *Probabilistic Robotics*: Bölüm 3 (Gauss filtreleri) ve Bölüm 5 (hareket modelleri). EKF türetimini buradan doğrula.
- Barfoot — *State Estimation for Robotics* (ileri seviye, hafta 5–6 için).
- ROS REP-105: `map / odom / base_link` çerçeve sözleşmesi — bonusa girmeden önce mutlaka oku.
- Kullandığın her sensörün resmî veri sayfası.

Son söz: Bu ödev bittiğinde “Kalman filtresi biliyorum” demeyeceksin;
 “Kalman filtresini *yazdım*, nerede kırıldığını gördüm, tamir ettim” diyeceksin.
 Fark, tam olarak BURST'ün aradığı fark. Kolay gelsin.